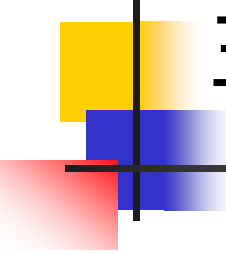


智能识别系统设计

上课地点：南区计算机大楼323

深圳大学
计算机与软件学院

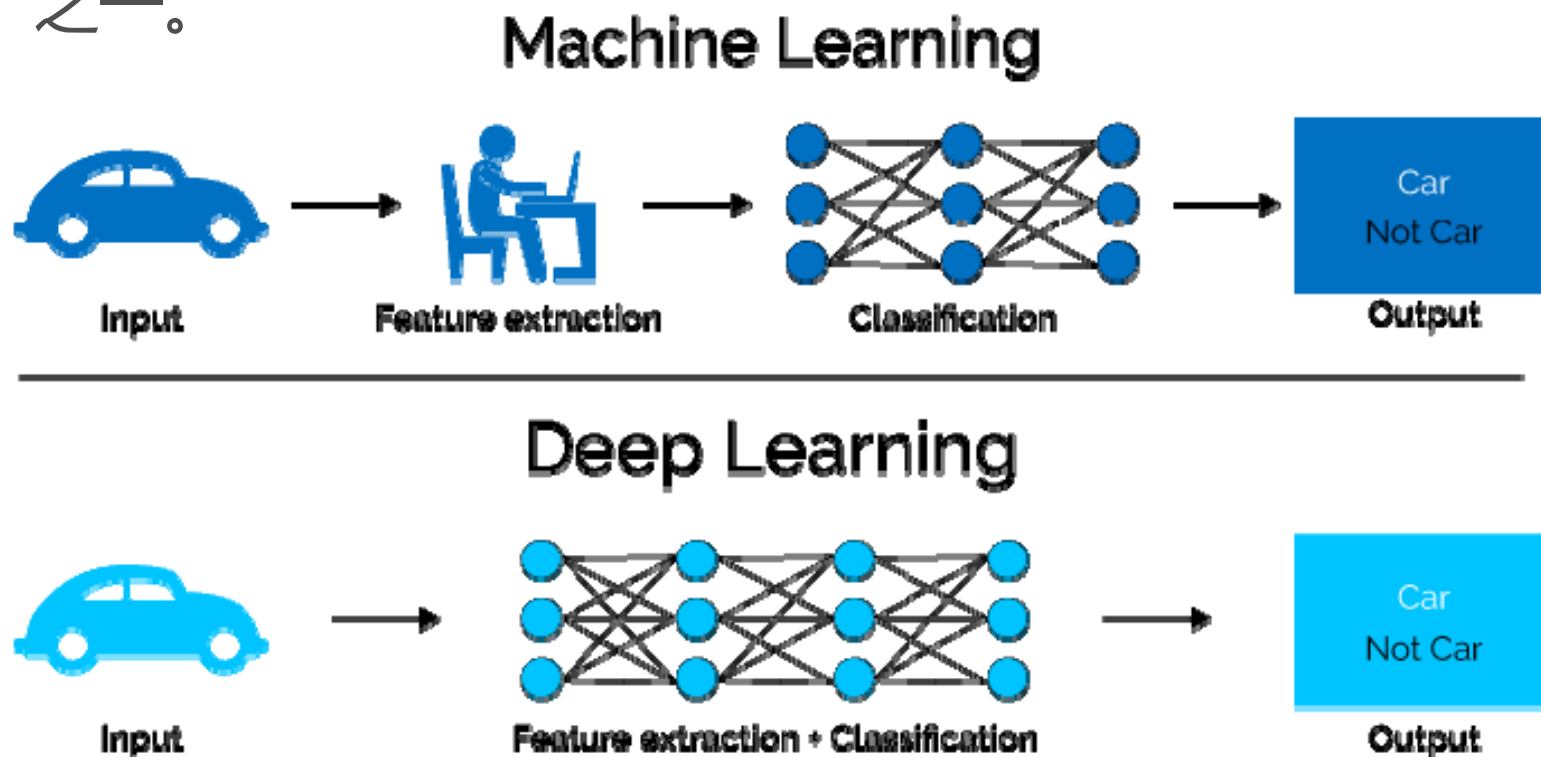


主要内容

- 1 深度学习基本概念
- 2 深度学习网络
- 3 深度学习平台
- 4 深度学习数据集

深度学习基本概念

深度学习(Deep Learning, DL) 是一种以人工神经网络为架构，对数据进行表征学习的方法。
深度学习是机器学习中一种基于对数据进行表征学习方法之一。

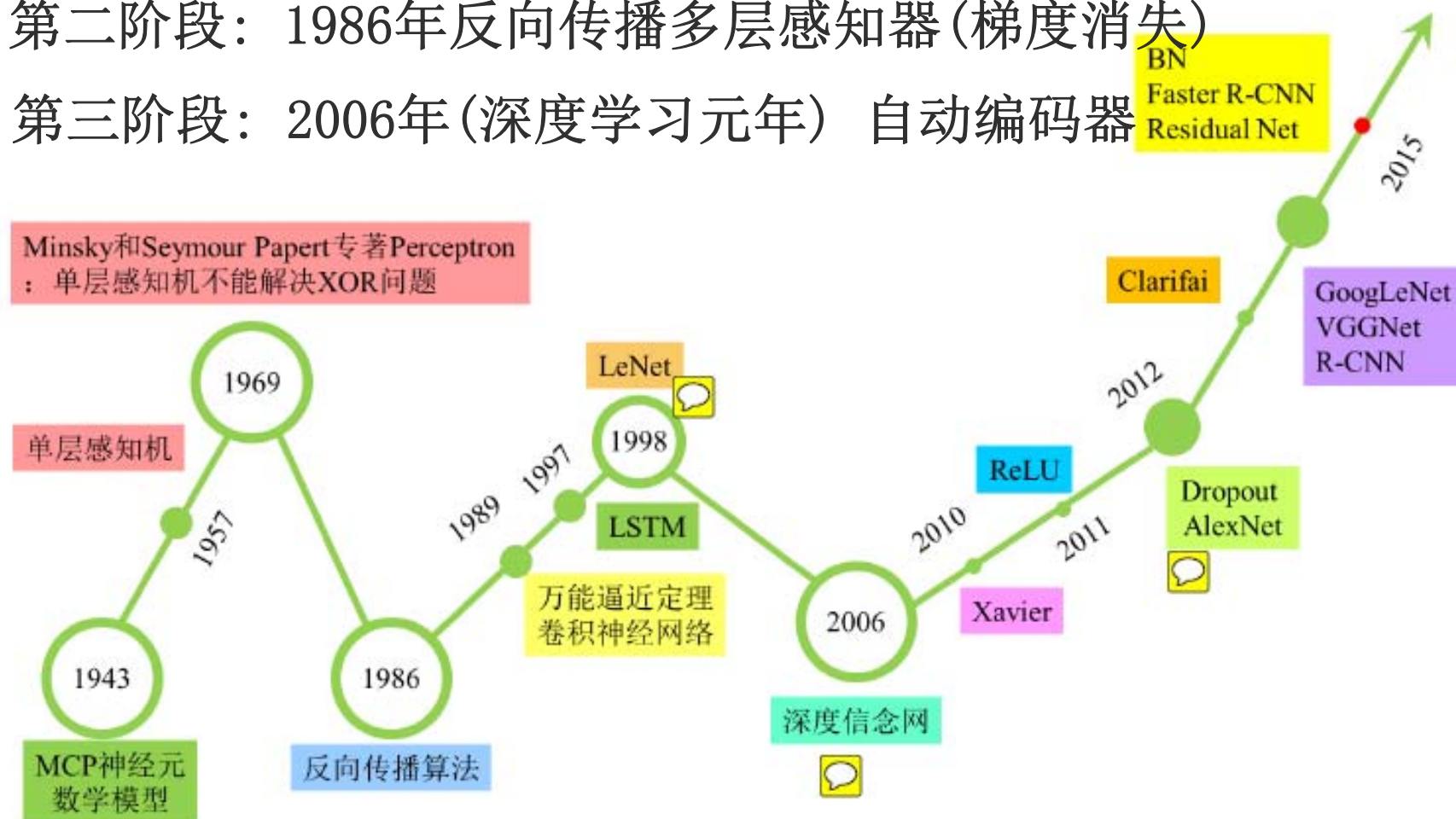


深度学习基本概念

第一阶段：1958年Rosenblatt发明的感知器(缺少非线性)

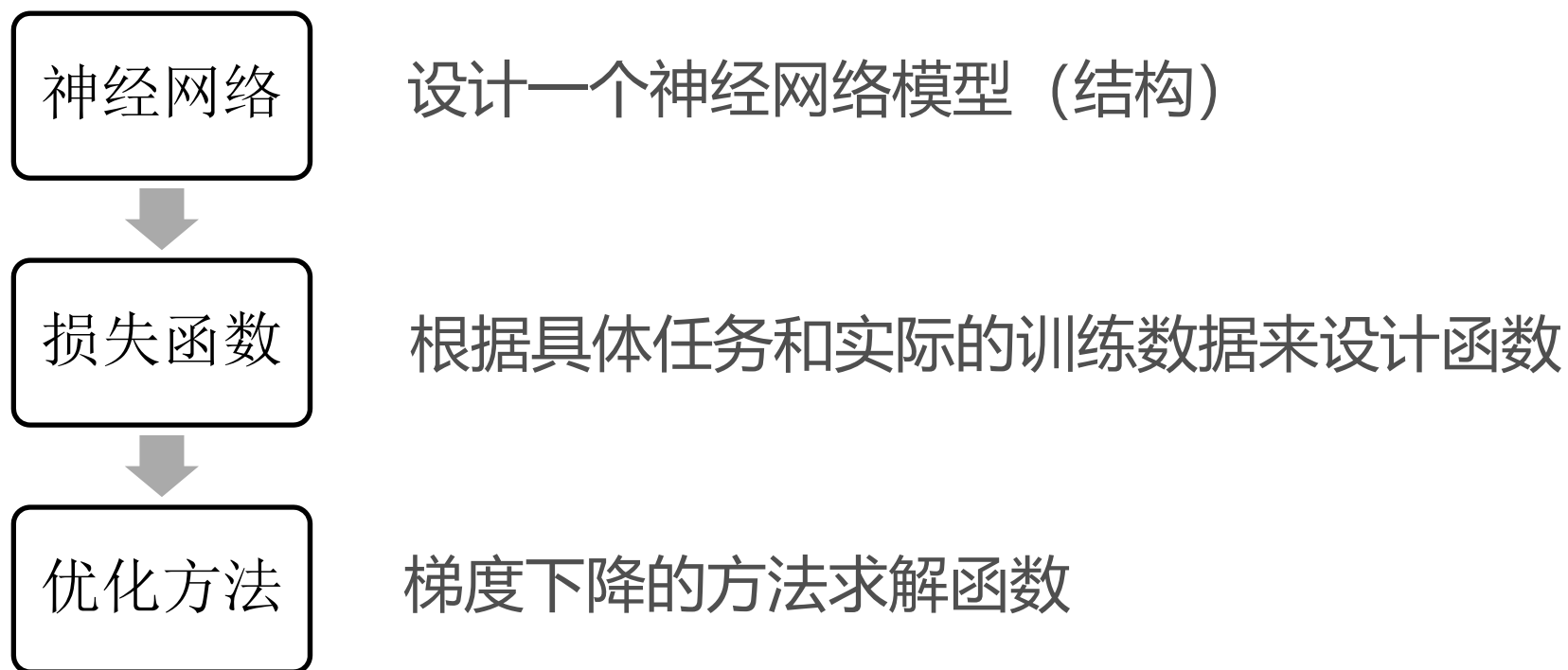
第二阶段：1986年反向传播多层感知器(梯度消失)

第三阶段：2006年(深度学习元年) 自动编码器



深度学习基本概念

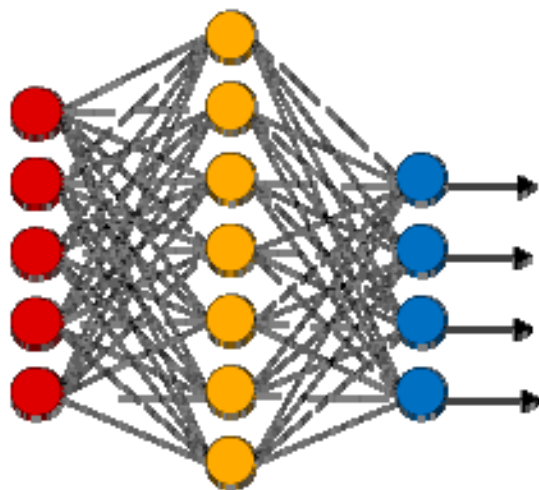
深度学习模型构建基本步骤:



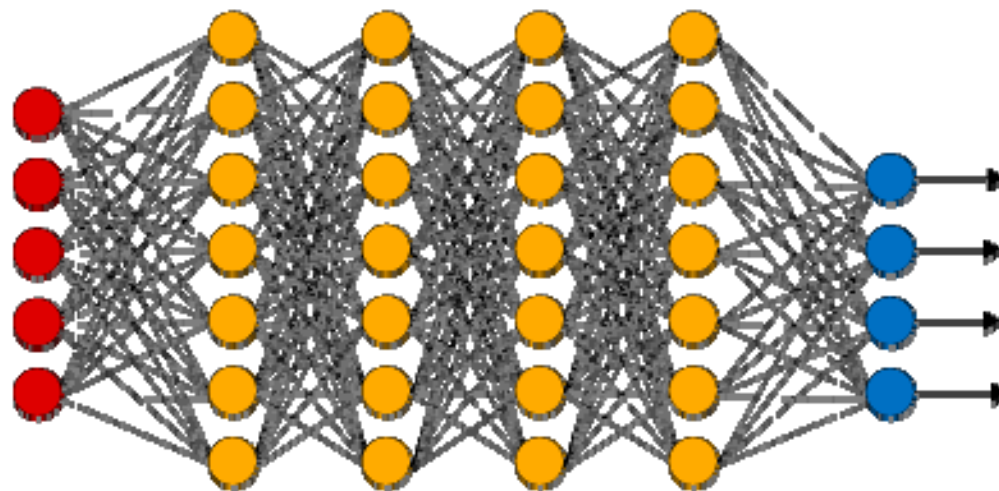
深度学习基本概念

深度学习模型示例：

Simple Neural Network



Deep Learning Neural Network



● Input Layer

● Hidden Layer

● Output Layer

深度学习网络

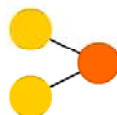
常见深度网络(CNN, GAN, RNN, GNN)

A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

- Backfed Input Cell
- Input Cell
- Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- Different Memory Cell
- Kernel
- Convolution or Pool

Perceptron (P)



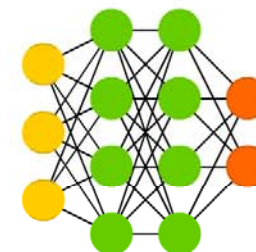
Feed Forward (FF)



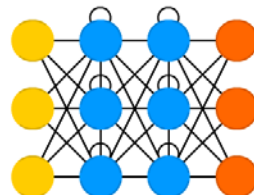
Radial Basis Network (RBF)



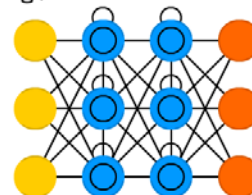
Deep Feed Forward (DFF)



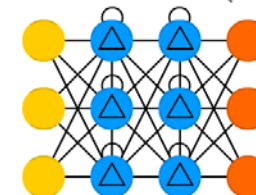
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)



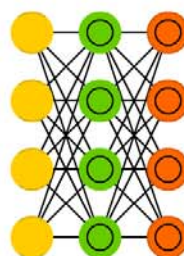
Gated Recurrent Unit (GRU)



Auto Encoder (AE)



Variational AE (VAE)



Denoising AE (DAE)



Sparse AE (SAE)



深度学习网络

Markov Chain (MC)



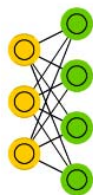
Hopfield Network (HN)



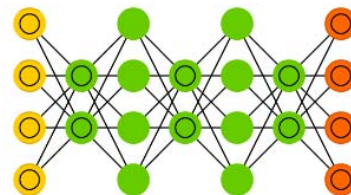
Boltzmann Machine (BM)



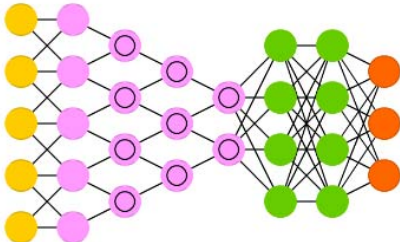
Restricted BM (RBM)



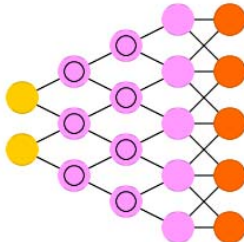
Deep Belief Network (DBN)



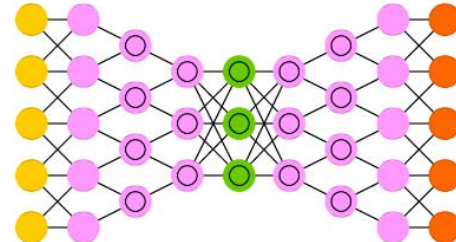
Deep Convolutional Network (DCN)



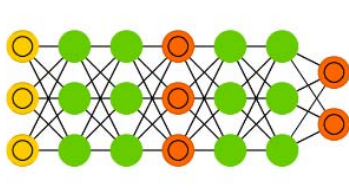
Deconvolutional Network (DN)



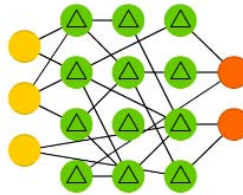
Deep Convolutional Inverse Graphics Network (DCIGN)



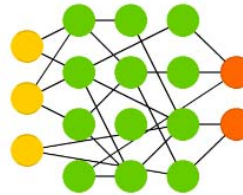
Generative Adversarial Network (GAN)



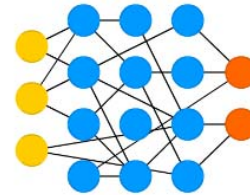
Liquid State Machine (LSM)



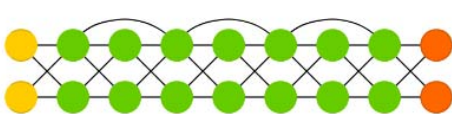
Extreme Learning Machine (ELM)



Echo State Network (ESN)



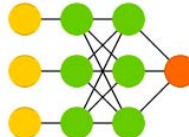
Deep Residual Network (DRN)



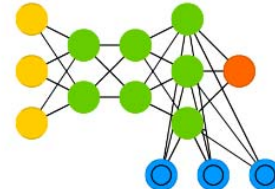
Kohonen Network (KN)



Support Vector Machine (SVM)



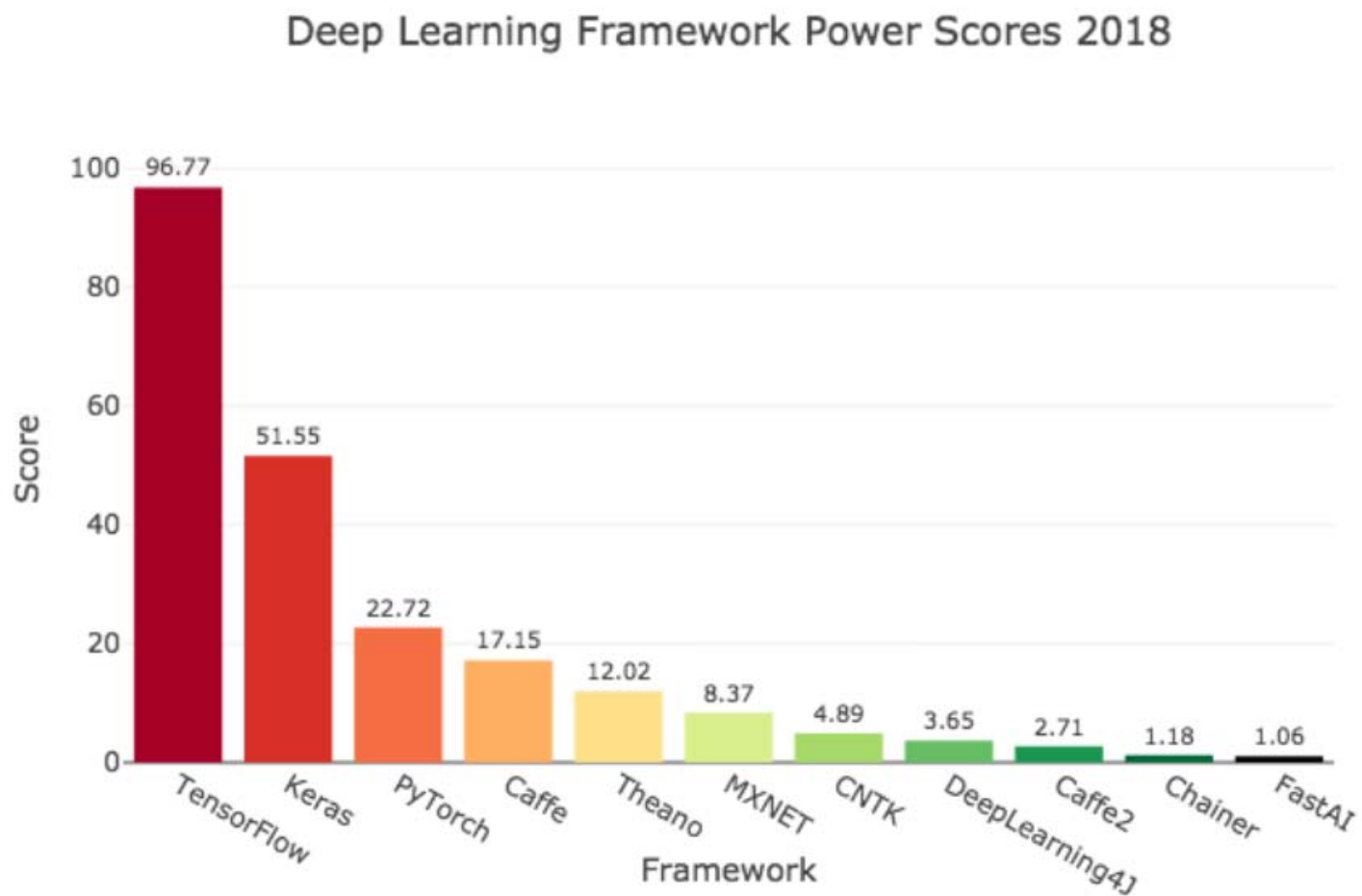
Neural Turing Machine (NTM)



<http://yann.lecun.com/exdb/lenet/index.htm>

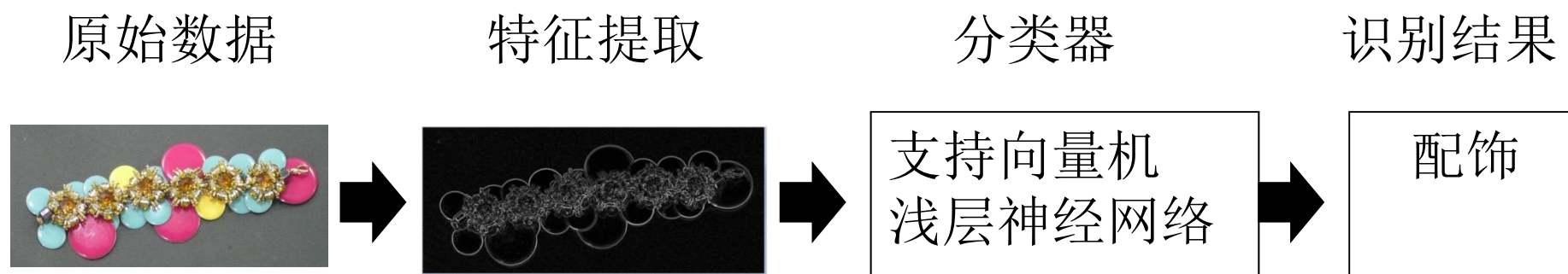
深度学习平台

A chart from 2018 showing some popular DL frameworks. Source: Hale 2018.*



● 卷积神经网络介绍

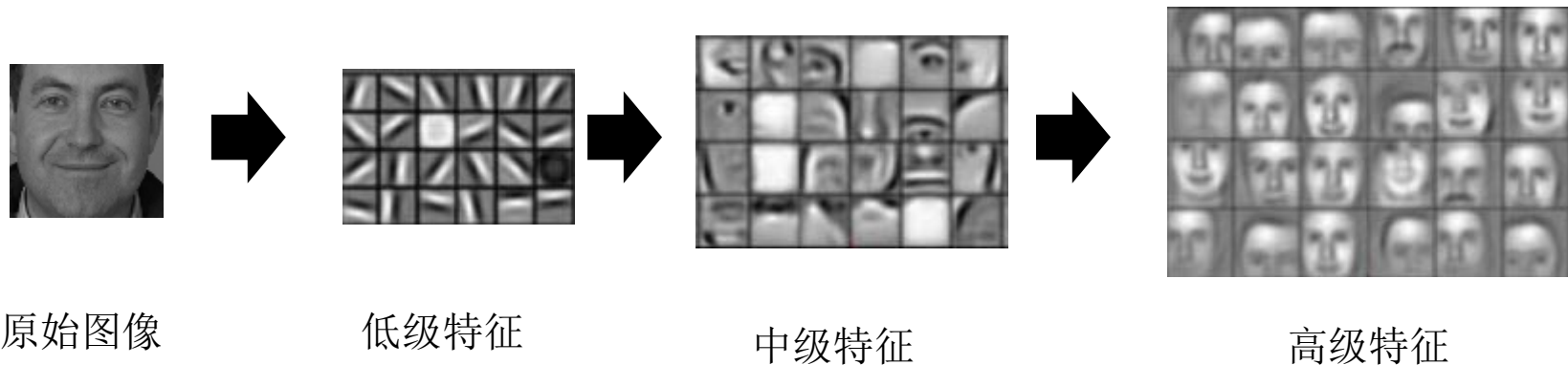
传统机器学习方法的局限



需要经验丰富的专家设计特征提取器!

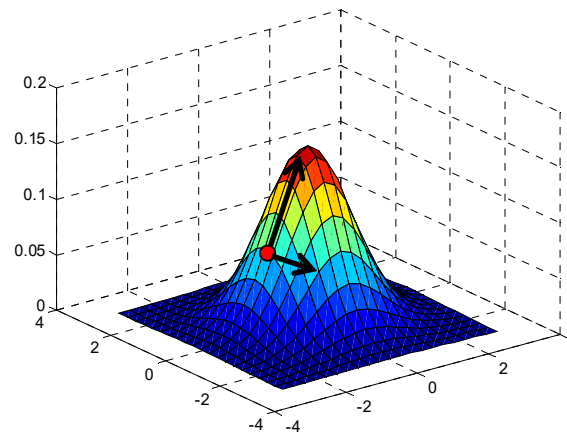
● 卷积神经网络介绍

- 深度学习方法不需要人工设计特征提取器；
- 深度学习是一种多层表示学习方法，用简单的线性模块构建而成，这些模块将上一层表示（从原始数据开始）转化为更高层、更抽象的表示。



● 卷积神经网络介绍

监督学习中，通过算法计算梯度向量调节权重向量（参数向量）



实际中，研究人员更多的使用随机梯度下降法，可以获得更低的损失。该方法包括输入少量样本、计算输出和误差、计算这些样本的平均梯度、根据梯度调节权值。

相比传统监督学习方法，深度学习可以自动学习特征、泛化性更好。

- 卷积神经网络介绍

反向传播算法

利用导数链式法则计算目标函数相对多层网络权值的梯度

● 卷积神经网络介绍

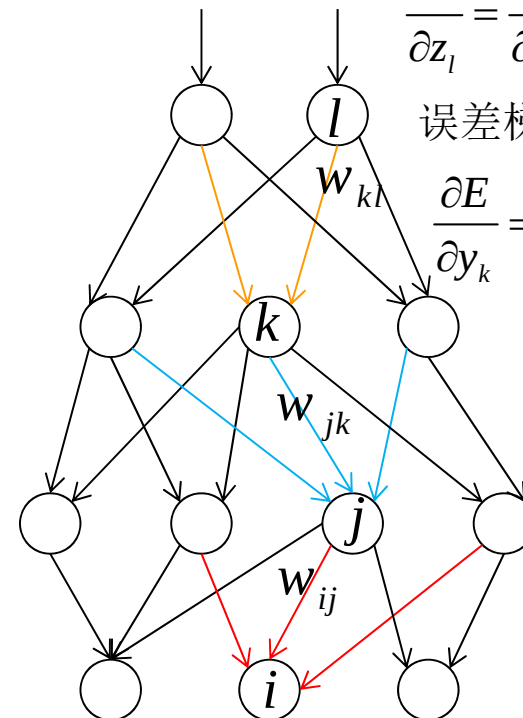
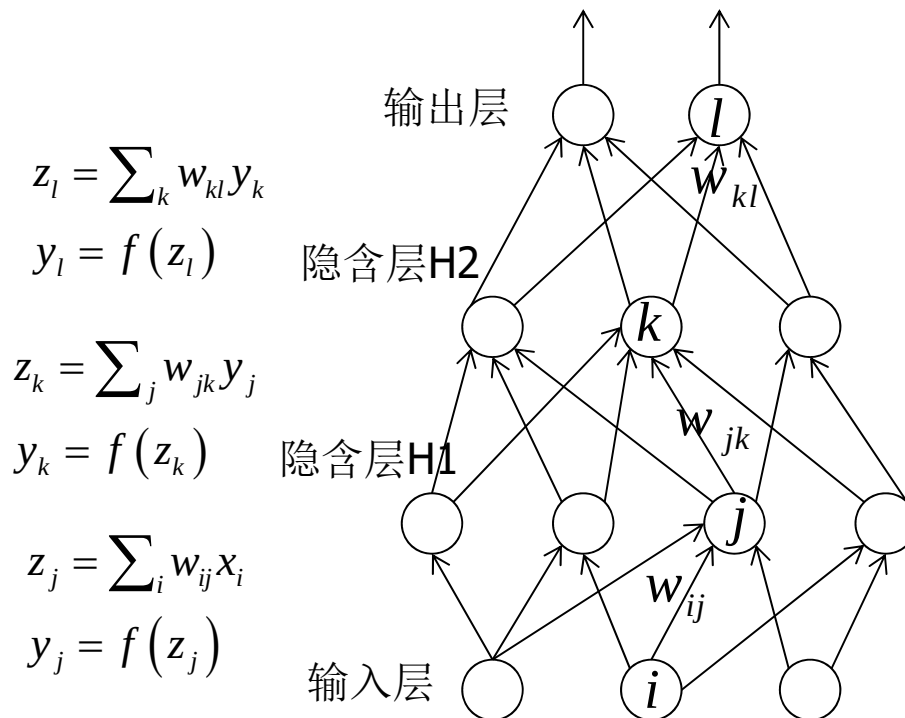
代价函数 $E = 0.5(y_l - t_l)^2$ $\frac{\partial E}{\partial y_l} = y_l - t_l$

$\frac{\partial E}{\partial z_l} = \frac{\partial E}{\partial y_l} \cdot \frac{\partial y_l}{\partial z_l} = (y_l - t_l) f'(z_l)$

误差梯度

$\frac{\partial E}{\partial y_k} = \sum_l \frac{\partial E}{\partial z_l} \cdot \frac{\partial z_l}{\partial y_k} = \sum_l w_{kl} \frac{\partial E}{\partial z_l}$

$\frac{\partial E}{\partial y_j} = \sum_k \frac{\partial E}{\partial z_k} \cdot \frac{\partial z_k}{\partial y_j}$
 $= \sum_k \frac{\partial E}{\partial y_k} \cdot \frac{\partial y_k}{\partial z_k} \cdot w_{jk}$
 $= \sum_k \frac{\partial E}{\partial y_k} \cdot f'(z_k) \cdot w_{jk}$

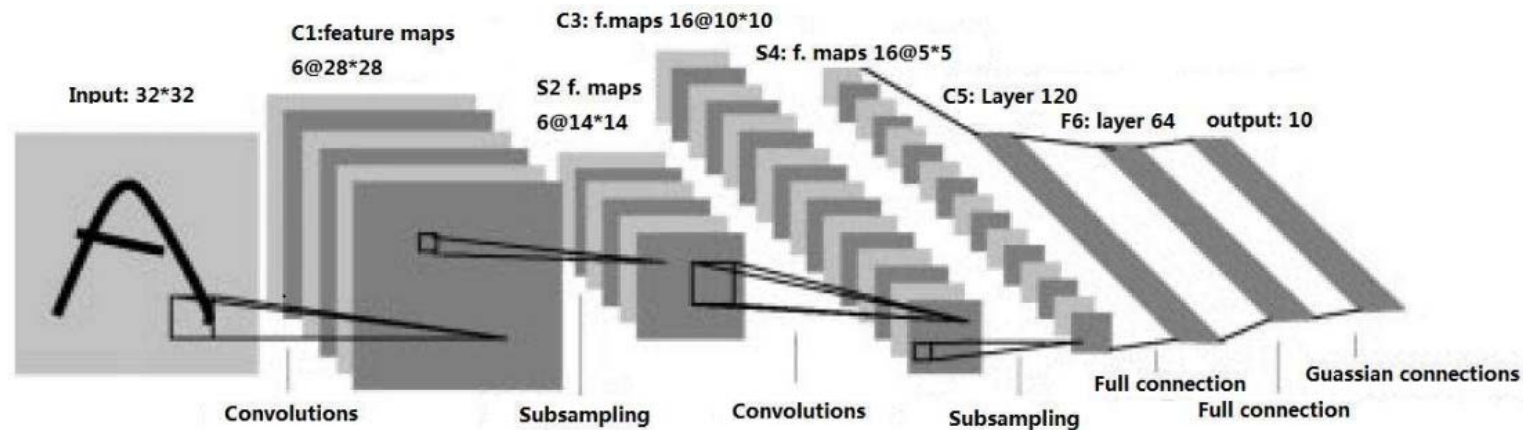


$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}} = \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j} \cdot x_i$ ←

$\frac{\partial E}{\partial x_i} = \sum_j \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial x_i} = \sum_j \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j} \cdot w_{ij}$

$w_{ij} \leftarrow w_{ij} + k \Delta w_{ij}$

● 卷积神经网络介绍



卷积神经网络（ConvNet）：前馈型网络，训练简单，泛化能力比相邻层用全连接更好

四个基本原则

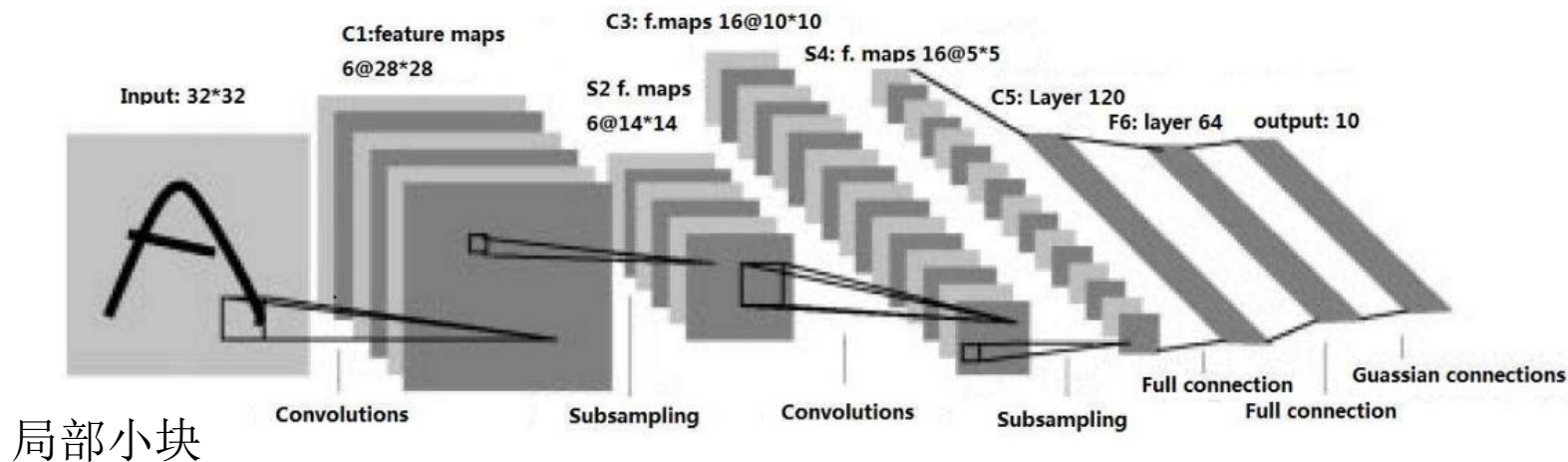
- 局部互联
- 共享权值：参数更少
- 下采样：保证局部不变性
- 多个卷积层：提取多样特征，训练时采用反向传播

● 卷积神经网络介绍

网络结构前几个阶段由两种层构成：卷积层和下采样

卷积层作用：检测前一层的局部连接特征

下采样作用：将语义相似的特征融合为一个，对平移、形变不敏感，保证新的特征层表示不敏感于前一层元素在位置和表观上的变化。

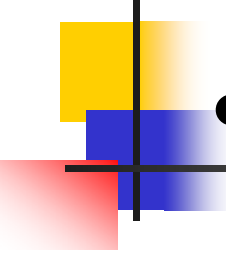


局部小块

特征图

滤波器组权值

局部小块的加权和被非线性单元（如ReLU）处理



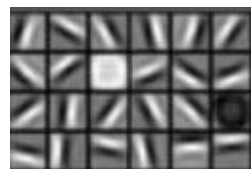
● 卷积神经网络介绍

- 同一特征图所有单元共享同一套滤波器组，而同一层不同特征图使用不同滤波器组。
- 如果一张图像存在某个基本图案，该图案可能出现在任意位置，那么不同位置单元共享相同的权值可以实现在数据的不同位置检测相同“模式”。

● 卷积神经网络介绍



原始图像



低级特征



中级特征



高级特征

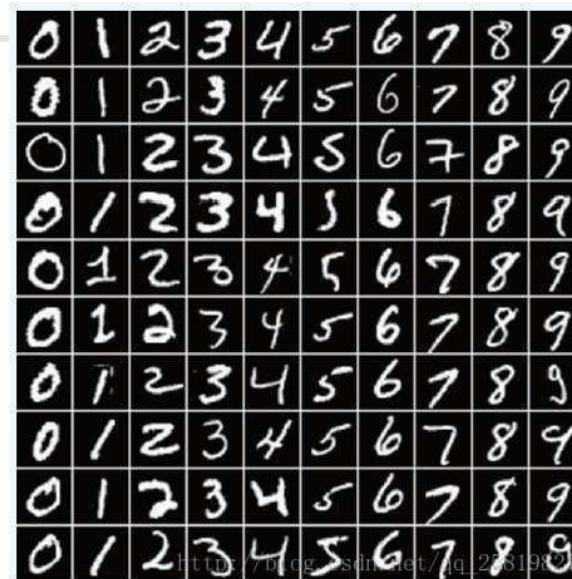
通过图示所提取的特征展示深度网络中的识别过程

深度学习数据集

(一) mnist手写字体数据集(分类):

mnist数据是由Google实验室的Corinna Cortes和纽约大学柯朗研究所的YannLeCun建立的一个手写字体数据集，其中训练集包含60000训练的手写数字图片，测试集包含10000张图片，一个训练集的标签集，一个测试集的标签集。

官方地址: <http://yann.lecun.com/exdb/mnist/>
数据集中图片为单通道，大小为28X28像素
基本上正确率都能达到98%+，适合初学者上手。



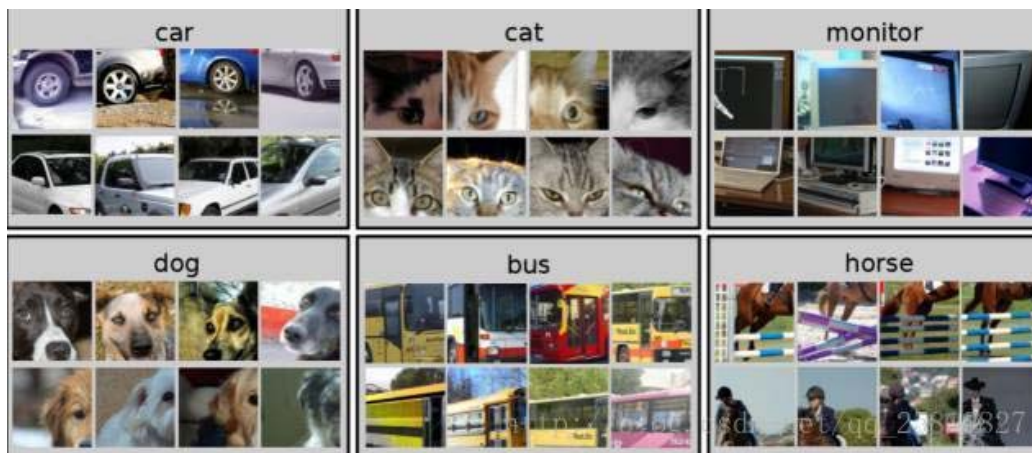
(二) ImageNet数据集(分类):

ImageNet数据是由Stanford University 李飞飞实验室主导发布，数据集中有1400万幅图像，涵盖了2万多个类别；其中有超过百万的图片有明确的类别标注和图像中物体位置的标注。



深度学习数据集

PASCAL VOC 分类识别与检测

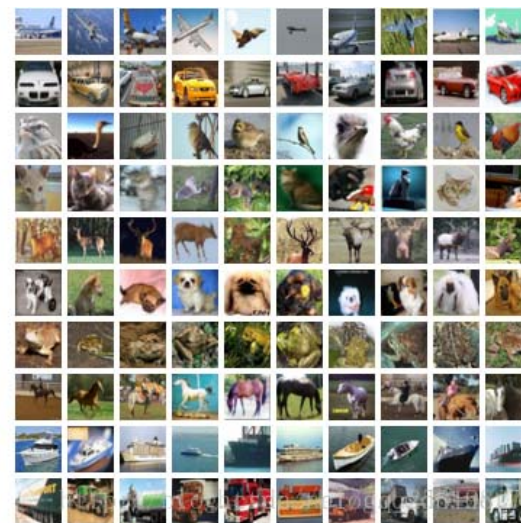


COCO-图像分割



cifar-100 分类

airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck



- 关于实验3和实验4

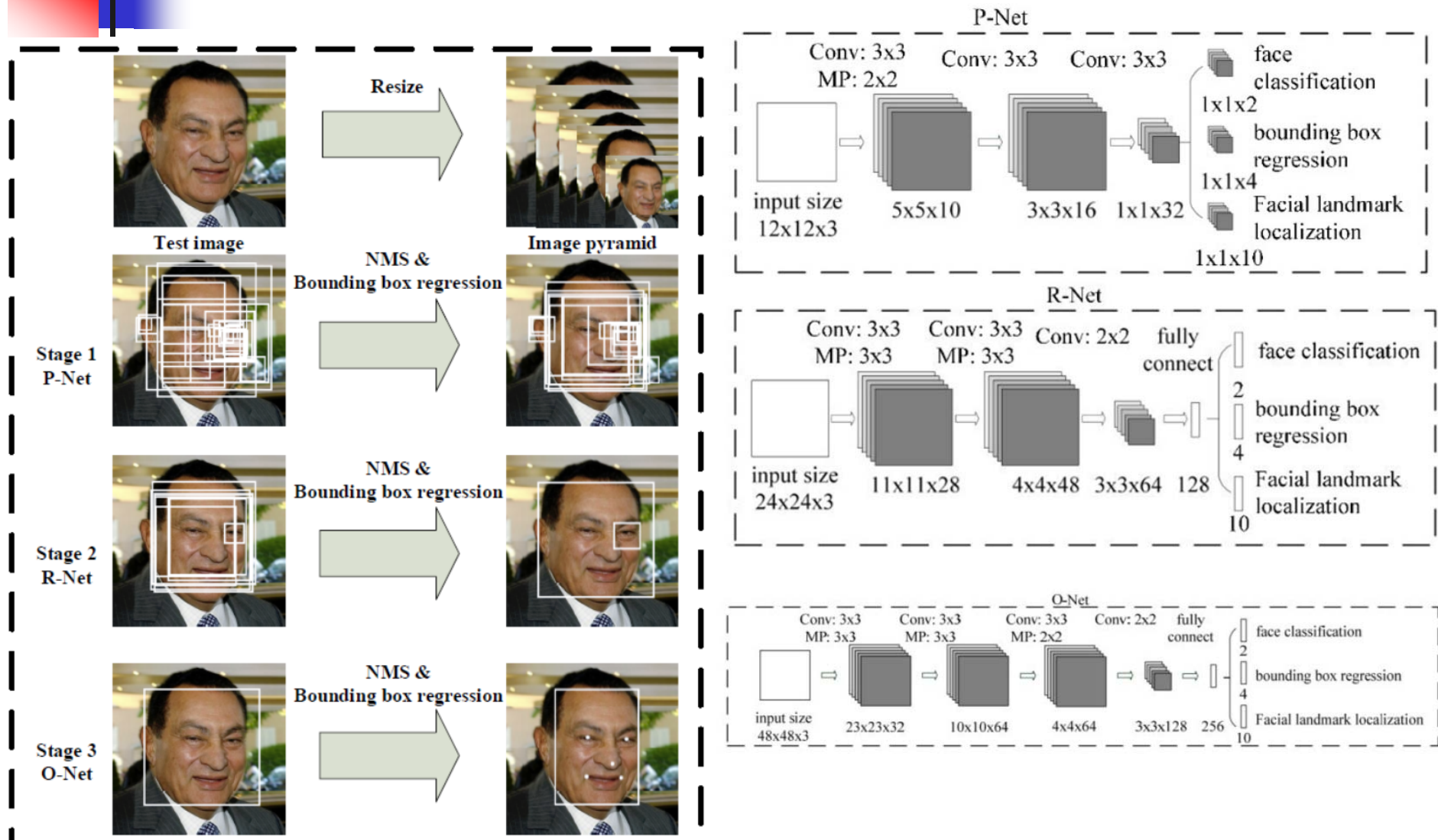
实验报告3内容-基于机器学习的目标检测实践

- MTCNN

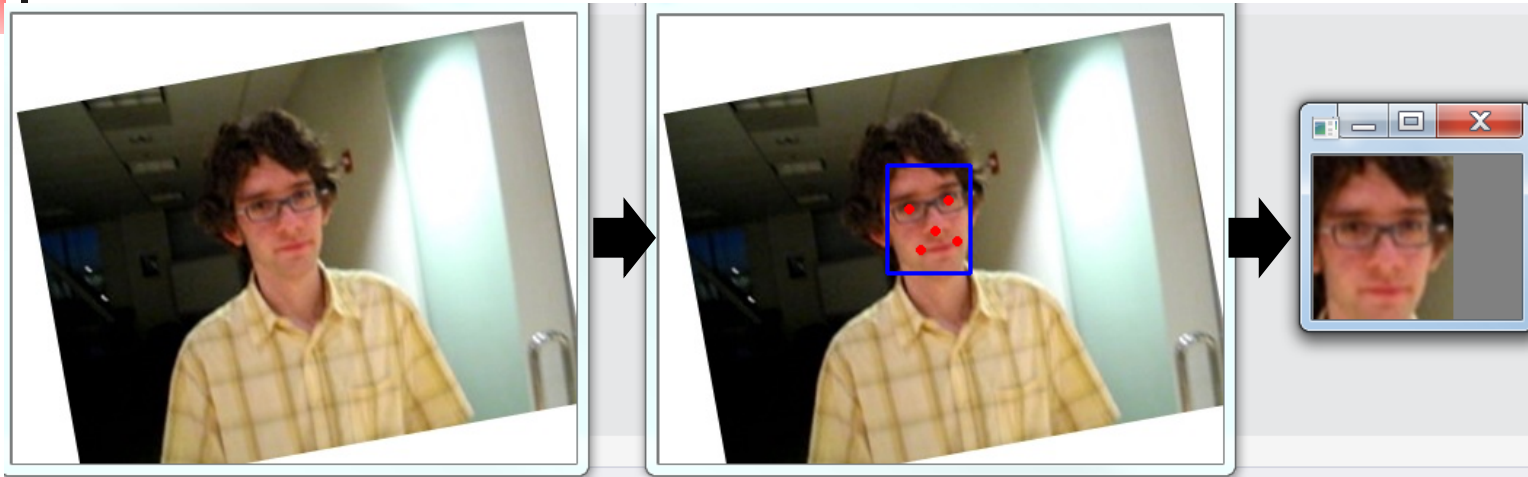
MTCNN: Kaipeng Zhang, Zhanpeng Zhang, Zhifeng Li, Yu Qiao, Multi-task Cascaded Convolutional Net works .

MTCNN是一种Multi-task的人脸检测框架，使用3个CNN级联算法结构，将人脸检测和人脸特征点检测同时进行

• MTCNN (Multi-task convolutional neural network)



Fast-MTCNN



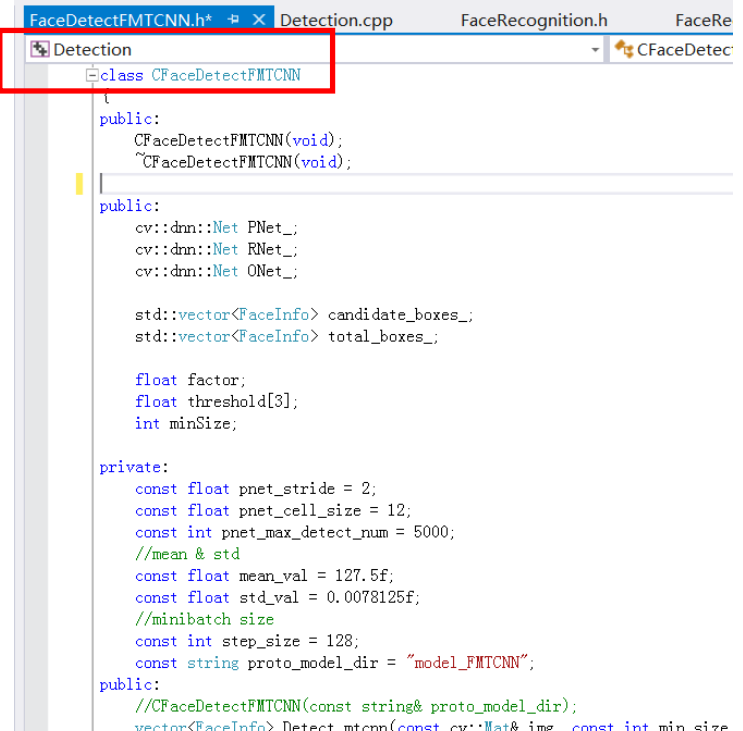
<https://github.com/szad670401/Fast-MTCNN>

```
int main(int argc, char **argv)
{
    MTCNN detector("model");
    string name_list[1] = {
        "test.jpg",
    };
    // MTCNN detector("./model");
    float factor = 0.709f;
    float threshold[3] = { 0.7f, 0.6f, 0.6f };
    int minSize = 12;
    for (int n = 0; n < 1; ++n){
        cv::Mat image = cv::imread(name_list[n], 1);

        for(int i = 0; i < 10; i++){
            double t = (double) cv::getTickCount();
            vector<FaceInfo> faceInfo = detector.Detect_mtcnn(image, minSize, threshold, factor, 3);
            std::cout << name_list[n] << " time," << (double) (cv::getTickCount() - t) / cv::getTickFrequency() << "s"
                << std::endl;

            for (int i = 0; i < faceInfo.size(); i++) {
                int x = (int) faceInfo[i].bbox.xmin;
                int y = (int) faceInfo[i].bbox.ymin;
                int w = (int) (faceInfo[i].bbox.xmax - faceInfo[i].bbox.xmin + 1);
                int h = (int) (faceInfo[i].bbox.ymax - faceInfo[i].bbox.ymin + 1);
                cv::rectangle(image, cv::Rect(x, y, w, h), cv::Scalar(255, 0, 0), 2);
            }
            cv::imwrite("test.png", image);
            cv::imshow("image", image);
            cv::waitKey(0);
        }
    }
}
```

Package FMTCNN into a Class.



```
class CFaceDetectFMTCNN
{
public:
    CFaceDetectFMTCNN(void);
    ~CFaceDetectFMTCNN(void);

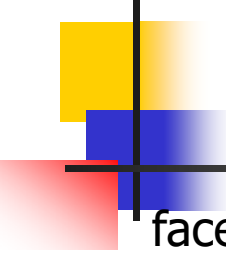
public:
    cv::dnn::Net PNet_;
    cv::dnn::Net RNet_;
    cv::dnn::Net ONet_;

    std::vector<FaceInfo> candidate_boxes_;
    std::vector<FaceInfo> total_boxes_;

    float factor;
    float threshold[3];
    int minSize;

private:
    const float pnet_stride = 2;
    const float pnet_cell_size = 12;
    const int pnet_max_detect_num = 5000;
    //mean & std
    const float mean_val = 127.5f;
    const float std_val = 0.0078125f;
    //minibatch size
    const int step_size = 128;
    const string proto_model_dir = "model_FMTCNN";

public:
    //CFaceDetectFMTCNN(const string& proto_model_dir);
    vector<FaceInfo> Detect_mtcnn(const cv::Mat& img, const int min_size,
```



```
faceInfo=Detect_mtcnn(image, minSize, threshold, factor, 3);
```

The main function returns:

(1) Face bounding box

(2) coordinates: Left eye X, Left eye Y, Right eye X, Right eye Y, Nose X, Noxe Y, Left Mouth Corner X, Left Mouth Corner Y, Right Mouth Corner X, Right Mouth Corner Y

```
typedef struct FaceBox {  
float xmin;  
float ymin;  
float xmax;  
float ymax;  
float score;  
} FaceBox;
```

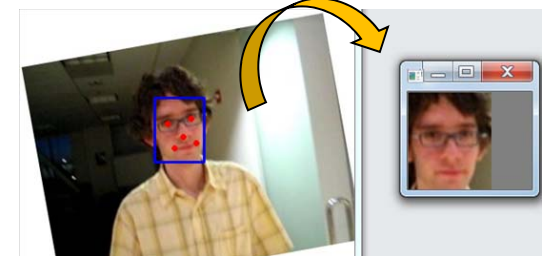
```
typedef struct FaceInfo {  
float bbox_reg[4];  
float landmark_reg[10]; //currently no value  
float landmark[10]; // coordinates: Left eye X, Left eye Y, Right eye X, Right eye Y, Nose X, Noxe Y, Left Mouth Corner X  
FaceBox bbox; //, Left Mouth Corner Y, Right Mouth Corner X, Right Mouth Corner Y  
cv::Mat FeatureVector; //store feature vector computed from dnn model  
} FaceInfo;
```

```
void CFaceDetectFMTCNN::RotateFace(cv::Mat image, FaceInfo faceInfoVar, cv::Mat &dstImage)
{
    //vector<FaceDetector::BoundingBox> res = fd.Detect(src, FaceDetector::BGR, FaceDetector::ORIENT_UP, 20, 0.6, 0.7, 0.7);
    //float std_points[10] = { 89.3095, 72.9025, 169.3095, 72.9025, 127.8949, 127.0441, 96.8796, 184.8907, 159.1065, 184.7601 };
    float std_points[10] = { 30.2946, 51.6963, 65.5318, 51.5014, 48.0252, 71.7366, 33.5493, 92.3655, 62.7299, 92.2041 };

    // 假设输入图片中只有一张人脸
    //printf("\n\n");
    float facial_points[10]; //坐标依次是: 左眼X, 左眼Y, 右眼X, 右眼Y, 鼻尖X, 鼻尖Y, 左嘴角X, 左嘴角Y, 右嘴角X, 右嘴角Y
    for (int i = 0; i < 5; i++) {
        facial_points[2*i] = faceInfoVar.landmark[2*i];
        facial_points[2*i+1] = faceInfoVar.landmark[2*i+1];
        //printf("%f, %f ", facial_points[2 * i], facial_points[2 * i + 1]);
    }
    //printf("\n");
    cv::Mat tform = getTformMatrix(std_points, facial_points); //这里得到的tform变换矩阵, 是w=3, h=2, 即2*3

    warpAffine(image, dstImage, tform, dstImage.size(), 1, 0, cv::Scalar(0));
}
```

Affine transform



```
void CFaceDetectFMTCNN::NormalizeFace(cv::Mat dstImage, cv::Mat &normalizeImg)
{
    cv::Mat subfactor = 127.5 * cv::Mat(IMG_NORHEIGHT, IMG_NORWIDTH, CV_32FC3, cv::Scalar(1, 1, 1));
    dstImage.convertTo(normalizeImg, CV_32FC3); //变为浮点型
    normalizeImg = normalizeImg - subfactor;
    normalizeImg = normalizeImg / 128; //归一化, 0-1
}
```

```

cv::Mat CFaceDetectFMTCNN::getTformMatrix(float* std_points, float* feat_points)
{
    int points_num_ = 5;
    double sum_x = 0, sum_y = 0;
    double sum_u = 0, sum_v = 0;
    double sum_xx_yy = 0;
    double sum_ux_vy = 0;
    double sum_vx_uy = 0;
    for (int c = 0; c < points_num_; ++c) {
        int x_off = c * 2;
        int y_off = x_off + 1;
        sum_x += std_points[c * 2];
        sum_y += std_points[c * 2 + 1];
        sum_u += feat_points[x_off];
        sum_v += feat_points[y_off];
        sum_xx_yy += std_points[c * 2] * std_points[c * 2] +
            std_points[c * 2 + 1] * std_points[c * 2 + 1];
        sum_ux_vy += std_points[c * 2] * feat_points[x_off] +
            std_points[c * 2 + 1] * feat_points[y_off];
        sum_vx_uy += feat_points[y_off] * std_points[c * 2] -
            feat_points[x_off] * std_points[c * 2 + 1];
    }
    double q = sum_u - sum_x * sum_ux_vy / sum_xx_yy
        + sum_y * sum_vx_uy / sum_xx_yy;
    double p = sum_v - sum_y * sum_ux_vy / sum_xx_yy
        - sum_x * sum_vx_uy / sum_xx_yy;
    double r = points_num_ - (sum_x * sum_x + sum_y * sum_y) / sum_xx_yy;
    double a = (sum_ux_vy - sum_x * q / r - sum_y * p / r) / sum_xx_yy;
    double b = (sum_vx_uy + sum_y * q / r - sum_x * p / r) / sum_xx_yy;
    double c = q / r;
    double d = p / r;
    cv::Mat Tinv = (cv::Mat_<float>(3, 3) << a, b, 0, -b, a, 0, c, d, 1);
    cv::Mat T = Tinv.inv();
    cv::Mat res = T.colRange(0, 2).clone();
    return res.t();
}

```

align_crop_example.cpp

Calculate affine transform matrix